

# Linked Lists

## 1 Introduction

A singly linked list is a data structure consisting of nodes, where each node contains data and a reference (or link) to the next node in the sequence. The last node has a reference to NULL, indicating the end of the list. Below are the basic operations that can be performed on a singly linked list.

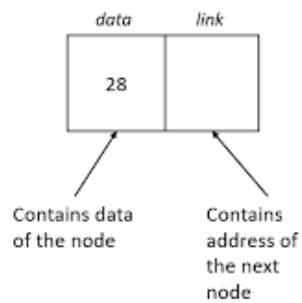


Figure 1: NODE of Linked List

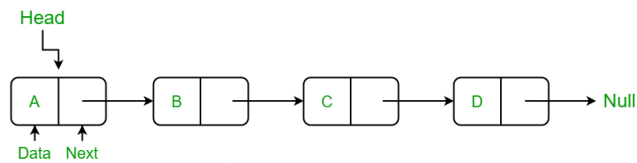


Figure 2: Creation of Singly Linked List

## 2 Operations on Singly Linked List

### 2.1 Creation of Singly Linked List

To create a singly linked list, a node structure is defined, typically containing a data field and a pointer to the next node. A linked list is initialized by setting the head pointer to NULL.

### 2.2 Addition of a Node

Nodes can be added to the linked list in various ways:

- **At the Beginning:** A new node is created, and its **next** pointer is set to the current head. The head pointer is then updated to point to the new node.
- **At the End:** A new node is created, and the list is traversed until the last node is found. The last node's **next** pointer is updated to point to the new node.
- **At a Specific Position:** The list is traversed until the node before the specified position is found. A new node is inserted between the nodes, and the pointers are adjusted accordingly.

### 2.3 Deletion of a Node

Nodes can be removed from the linked list by adjusting the pointers:

- **From the Beginning:** The head pointer is updated to point to the second node, and the first node is deleted.
- **From the End:** The list is traversed to find the second-to-last node, whose **next** pointer is set to NULL. The last node is then deleted.
- **From a Specific Position:** The list is traversed to find the node before the one to be deleted. Pointers are adjusted to skip the deleted node.

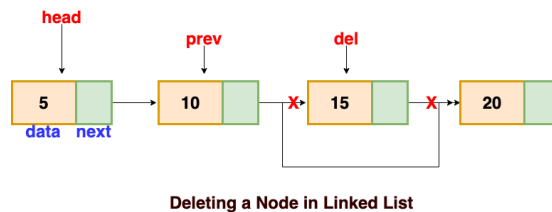


Figure 3: deleting an element

## 2.4 Finding an Element

To find an element, the list is traversed from the head node, comparing each node's data with the target value. If a match is found, the position is returned; otherwise, the search continues until the end of the list.

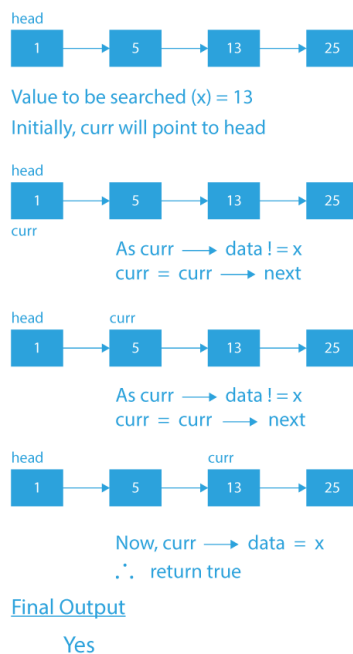


Figure 4: finding an element

## 2.5 Traversing and Printing the List

Traversing involves iterating through each node in the list, starting from the head, and visiting each node until the end of the list is reached. During traversal, the data of each node can be printed.

## 2.6 Reversing the List

Reversing a linked list involves changing the direction of the pointers. This can be done by iterating through the list and reversing the **next** pointers of each node, so that each node points to the previous node instead.

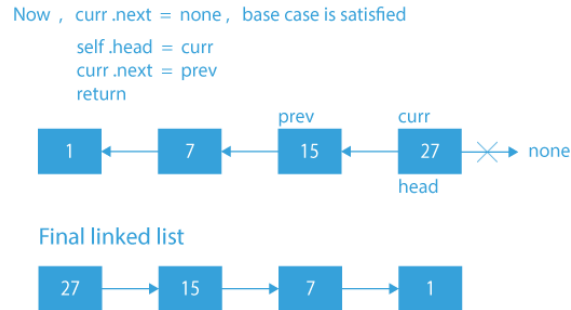


Figure 5: reversing linkedLits

## 2.7 Swapping Nodes Based on Positions

Swapping nodes based on positions involves identifying the nodes at the specified positions and adjusting the pointers to swap them without changing the data within the nodes. Care must be taken when swapping adjacent nodes or when the nodes involve the head or tail of the list.

## 3 Conclusion

These operations provide basic functionalities for managing and manipulating singly linked lists. Understanding these operations is essential for working with linked lists and implementing more complex data structures.